```
  def load_task
    @task = Task.find(params[:id])
  end

  def task_params
    params.require(:task).permit(:project_id, :title, :size)
  end
end
```

I still don't think the controller logic warrants another test.

Now, in the app/views/projects/show.html.erb template file, replace the table-row line with the following:

```
<tr id="task_<%= task.project_order %>">
```

That gives you the #task_3 selector, at long last.

And that brings me to the next point of failure, Unable to find link or button with text "Up", meaning that the update logic is not yet in place.

What's the logic you want? And how can you write a test for it?

You want an "Up" link for all tasks but the first one, and a "Down" link for all tasks but the last one. That means you need to be able to tell if a task is first or last. That's testable:

integration/02/spec/models/task_spec.rb

```
describe "order", aggregate_failures: true do
  let(:project) { create(:project, name: "Project") }
  let!(:first) { project.tasks.create!(project_order: 1) }
  let!(:second) { project.tasks.create!(project_order: 2) }
  let!(:third) { project.tasks.create!(project_order: 3) }

  it "can determine that a task is first or last" do
    expect(first).to be_first_in_project
    expect(first).not_to be_last_in_project
    expect(second).not_to be_first_in_project
    expect(second).not_to be_last_in_project
    expect(third).not_to be_first_in_project
    expect(third).to be_last_in_project
  end

  it "knows neighbors" do
    expect(second.previous_task).to eq(first)
    expect(second.next_task).to eq(third)
    expect(first.previous_task).to be_nil
    expect(third.next_task).to be_nil
  end
```